

TP 2 : sécurité du réseau et protection des données

Module Sécurité Cloud - Durée 3 h - Travail en trinome

Objectif

Le CM de ce matin a posé deux principes : par défaut rien n'est joignable, et une donnée se protège même quand le réseau échoue. Ce TP vous permettra de les mettre en pratique. Vous allez construire un réseau cloisonné, y introduire puis détecter une exposition, écrire l'outil qui la détecte, puis protéger des données par chiffrement à clé enveloppée et verrouiller un stockage objet.

À la fin de la séance, vous devriez savoir répondre à trois questions devant un architecte : qu'est-ce qui est joignable depuis Internet et pourquoi, qui peut lire cette donnée si le fichier est volé, et comment le prouver sans faire confiance aux noms des objets.

Cadre

- Travail limité aux ressources fournies dans ce sujet et à votre poste. Toute tentative d'accès à un système tiers sort du cadre pédagogique et relève de l'article 323-1 et suivants du Code pénal.
- Rendu individuel par binôme : un compte rendu au format PDF, plus les fichiers produits. Voir la section « Rendu » ci-dessous.
- Les commandes de ce sujet ont été exécutées et vérifiées. Si l'une échoue, signalez-le : c'est probablement un écart d'environnement, pas une faute de votre part.

Rendu

Un dossier compressé contenant :

- le compte rendu en PDF, huit pages maximum, avec la fiche de relevé complétée, les sorties de commande demandées et les réponses rédigées ;
- quelques captures d'écran
- les trois fichiers produits : `audit_reseau.py` complété, `coffre.py` et `tls-obligatoire.json` ;
- les deux sorties de l'audit réseau, avant et après correction.

Aucun identifiant réel ne doit figurer dans le rendu, même de l'émulateur.

Mise en place de l'environnement

Principe

Le dispositif est le même que pour le TP1 : si vous avez conservé cet environnement, activez-le et passez directement à l'étape 2, en remplaçant partout `tp2-reseau` par le nom du vôtre. La bibliothèque `boto3`, dont vous aurez besoin aujourd'hui, y est déjà installée avec `moto`.

`moto` simule localement les services d'AWS : votre poste répond à la place du fournisseur, sans compte, sans facturation et sans requête sortante. C'est l'outil que les équipes utilisent couramment pour tester du code d'infrastructure sans toucher à un environnement réel.

Prérequis unique : **Python 3.10 ou supérieur**. Vérifiez-le avant toute chose avec `python3 --version` (Linux, macOS) ou `py -3 --version` (Windows). Sous macOS, si la commande propose d'installer les outils en ligne de commande, acceptez ; sinon installez Python depuis python.org.

La recherche sur Internet est autorisée pendant la séance, documentation du fournisseur comprise.

Étape 1 : créer l'environnement et installer les outils

Linux (Ubuntu 24.04) et macOS

```
python3 -m venv tp2-reseau
source tp2-reseau/bin/activate
pip install "moto[server]" awscli
```

Windows, dans l'invite de commandes `cmd.exe` :

```
py -3 -m venv tp2-reseau
tp2-reseau\Scripts\activate.bat
pip install "moto[server]" awscli
```

En PowerShell, l'activation est `tp2-reseau\Scripts\Activate.ps1`, précédée si besoin de `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`, qui ne vaut que pour la fenêtre en cours et ne demande aucun droit administrateur.

Étape 2 : démarrer le serveur

Dans ce terminal, l'environnement étant activé, lancez le serveur et **laissez la fenêtre ouverte** pendant toute la séance :

```
moto_server -p 5055
```

La fenêtre doit afficher `Running on http://127.0.0.1:5055`.

Étape 3 : préparer le second terminal

Ouvrez un **second** terminal, placez-vous dans le dossier qui *contient* votre environnement (et non dans le dossier de l'environnement lui-même), activez-le avec la même commande qu'à l'étape 1, puis déclarez quatre variables.

Linux et macOS

```
export AWS_ACCESS_KEY_ID=test
export AWS_SECRET_ACCESS_KEY=test
export AWS_DEFAULT_REGION=eu-west-3
export AWS_ENDPOINT_URL=http://localhost:5055
```

Windows, `cmd.exe` : remplacez `export` par `set`. **Windows**, PowerShell :

```
$env:AWS_ACCESS_KEY_ID="test"
$env:AWS_SECRET_ACCESS_KEY="test"
$env:AWS_DEFAULT_REGION="eu-west-3"
$env:AWS_ENDPOINT_URL="http://localhost:5055"
```

C'est la seule différence entre les trois systèmes. Grâce à la variable `AWS_ENDPOINT_URL`, toutes les commandes du sujet s'écrivent ensuite exactement de la même façon partout.

Étape 4 : vérifier

```
aws sts get-caller-identity
```

Vous devez obtenir un bloc JSON contenant "Account": "123456789012". Si la commande répond `Could not connect to the endpoint URL`, le serveur n'est pas démarré; si elle répond `command not found` ou `aws n'est pas reconnu`, l'environnement n'est pas activé dans ce terminal.

Conventions du sujet

- Les fichiers JSON et Python sont donnés en clair et sont à créer avec votre éditeur de texte, dans le dossier courant.
- Les identifiants de ressources (`vpc-...`, `subnet-...`) sont **tirés au hasard à la création**. Relevez-les au fur et à mesure dans la fiche de l'annexe A, et remplacez les `xxxx` du sujet par les vôtres. C'est volontaire : apprendre à lire un identifiant fait partie du métier.
- Chaque commande tient sur une seule ligne.
- L'annexe E rassemble toutes les commandes du sujet : gardez-la sous les yeux plutôt que de remonter dans le texte.

Ce que l'émulateur fait, et ce qu'il ne fait pas

À lire avant de commencer : L'émulateur reproduit fidèlement les API : création de VPC, subnets, tables de routage, groupes de sécurité, clés KMS, buckets et secrets, avec leurs attributs. En revanche, **aucun paquet ne circule** et les politiques ne sont pas évaluées : vous ne verrez ni connexion refusée, ni accès bloqué.

Ce n'est pas une limite gênante, à condition d'en tirer la bonne conclusion : l'exposition d'une ressource se démontre en lisant sa configuration, pas en tentant une connexion. C'est exactement ce que fait un outil de posture, et c'est ce que vous écrirez à l'exercice 3.

1 Construire un réseau cloisonné

Vous déployez le plan d'adressage vu en cours, réduit à une seule zone de disponibilité. La seconde zone serait identique.

Nom	CIDR	Rôle	Route par défaut attendue
public-a	10.0.0.0/24	entrée Internet	vers l'Internet Gateway
app-a	10.0.10.0/24	applicatif	aucune
data-a	10.0.20.0/24	données	aucune

1. Créez le VPC, puis relevez son identifiant :

```
aws ec2 create-vpc --cidr-block 10.0.0.0/16
```

2. Créez les trois subnets, en remplaçant `vpc-xxxx` par votre identifiant, et relevez les trois identifiants obtenus :

```
aws ec2 create-subnet --vpc-id vpc-xxxx --cidr-block 10.0.0.0/24 --availability-zone eu-west-3a
aws ec2 create-subnet --vpc-id vpc-xxxx --cidr-block 10.0.10.0/24 --availability-zone eu-west-3a
aws ec2 create-subnet --vpc-id vpc-xxxx --cidr-block 10.0.20.0/24 --availability-zone eu-west-3a
```

3. Nommez-les, car l'exercice 3 lira ces étiquettes :

```
aws ec2 create-tags --resources subnet-xxxx --tags Key=Name,Value=public-a
aws ec2 create-tags --resources subnet-yyyy --tags Key=Name,Value=app-a
aws ec2 create-tags --resources subnet-zzzz --tags Key=Name,Value=data-a
```

4. Créez la passerelle Internet et attachez-la au VPC :

```
aws ec2 create-internet-gateway
aws ec2 attach-internet-gateway --internet-gateway-id igw-xxxx --vpc-id vpc-xxxx
```

5. Créez la table de routage *publique*, ajoutez-lui la route par défaut, puis associez-la au seul subnet *public-a* :

```
aws ec2 create-route-table --vpc-id vpc-xxxx
aws ec2 create-route --route-table-id rtb-xxxx --destination-cidr-block 0.0.0.0/0 --gateway-id igw-xxxx
aws ec2 associate-route-table --route-table-id rtb-xxxx --subnet-id subnet-xxxx
```

6. Créez une seconde table de routage, sans aucune route ajoutée, et associez-la au seul subnet *app-a*. Laissez volontairement *data-a* sans association explicite : vous verrez à l'exercice 3 ce que cela change.
7. Vérifiez votre travail et recopiez la sortie dans le compte rendu :

```
aws ec2 describe-route-tables --filters Name=vpc-id,Values=vpc-xxxx
```

8. Question : la table de routage créée automatiquement avec le VPC, dite *principale*, contient déjà une route. Laquelle, et pourquoi ne peut-elle pas être supprimée ? Quel subnet utiliserait cette table si vous aviez oublié une association ?

2 Filtrage en cascade

Trois groupes de sécurité, un par tiers, chacun n'acceptant que le tiers précédent.

1. Créez les trois groupes et relevez leurs identifiants :

```
aws ec2 create-security-group --group-name sg-alb --description "entree publique" --vpc-id vpc-xxxx
aws ec2 create-security-group --group-name sg-app --description "tier applicatif" --vpc-id vpc-xxxx
aws ec2 create-security-group --group-name sg-db --description "base de donnees" --vpc-id vpc-xxxx
```

2. Ouvrez le seul flux entrant légitime depuis Internet :

```
aws ec2 authorize-security-group-ingress --group-id sg-alb-xxxx --protocol tcp --port 443 --cidr 0.0.0.0/0
```

3. Chaînez les deux tiers suivants **par référence**, sans jamais écrire d'adresse IP :

```
aws ec2 authorize-security-group-ingress --group-id sg-app-xxxx --protocol tcp --port 8443 --source-group sg-alb-xxxx
aws ec2 authorize-security-group-ingress --group-id sg-db-xxxx --protocol tcp --port 5432 --source-group sg-app-xxxx
```

4. Un collègue pressé ajoute une règle pour déboguer. Exécutez-la : c'est le défaut que vous détecterez à l'exercice suivant.

```
aws ec2 authorize-security-group-ingress --group-id sg-app-xxxx --protocol tcp --port 22 --cidr 0.0.0.0/0
```

5. Question : pourquoi la règle par référence de la question 3 est-elle plus robuste qu'une règle par plage d'adresses, dans un environnement où le nombre d'instances varie automatiquement ?

3 Écrire un mini-outil de posture

Aucun paquet ne circule, et pourtant vous devez prouver ce qui est exposé. C'est le principe d'un outil de posture : il lit la configuration et calcule l'accessibilité.

1. Créez le fichier `audit_reseau.py` avec le contenu de l'annexe B. Deux fonctions sont volontairement incomplètes, repérées par `# A COMPLETER`.
2. Complétez la première : un subnet est exposé si la table de routage qui lui est associée contient une route `0.0.0.0/0` dont la cible commence par `igw-`.
3. Complétez la seconde : une règle est à signaler si elle autorise `0.0.0.0/0` sur le port 22 ou 3389.
4. Exécutez l'audit sur votre VPC :

```
python audit_reseau.py vpc-xxxx
```

Vous devez obtenir trois informations : un subnet exposé, un subnet signalé « à vérifier » faute d'association explicite, et la règle fautive de l'exercice précédent.

5. Corrigez les deux défauts : révoquez la règle, et associez explicitement `data-a` à la table de routage privée. Relancez l'audit et montrez les deux sorties, avant et après, dans le compte rendu.

```
aws ec2 revoke-security-group-ingress --group-id sg-app-xxxx --protocol tcp --port 22 --cidr 0.0.0.0/0
aws ec2 associate-route-table --route-table-id rtb-prive-xxxx --subnet-id subnet-data-xxxx
```

6. Question : avant correction, `data-a` n'était associé à aucune table et utilisait donc la table principale du VPC. Expliquez pourquoi cette situation est risquée en production, même lorsque la table principale ne contient aucune route vers Internet, et ce qu'un outil professionnel devrait vérifier de plus que le vôtre.

4 Chiffrement à clé enveloppée

Le réseau peut échouer, la donnée doit rester illisible. Vous implémentez le mécanisme vu en cours : une clé maîtresse chez le fournisseur, une clé de données jetable qui chiffre localement.

1. Créez la clé maîtresse et son alias :

```
aws kms create-key --description "cle maitresse TP2"
aws kms create-alias --alias-name alias/tp2 --target-key-id VOTRE-KEY-ID
```

2. Observez ce que renvoie la demande d'une clé de données. Deux formes de la même clé apparaissent : en clair, et chiffrée par la clé maîtresse.

```
aws kms generate-data-key --key-id alias/tp2 --key-spec AES_256
```

3. Créez `coffre.py` avec le contenu de l'annexe C, puis exécutez-le :

```
python coffre.py
```

4. Ouvrez le fichier `coffre.bin` produit, avec un éditeur de texte ou avec la commande suivante. Le message d'origine y est-il lisible ?

```
python -c "print(open('coffre.bin', 'rb').read()[:60])"
```

5. Questions à traiter :

- Pourquoi la clé maîtresse ne chiffre-t-elle pas directement le fichier ? Donnez l'argument de performance *et* l'argument de sécurité.

- Quelle partie du fichier `coffre.bin` devient inutilisable si la clé maîtresse est désactivée ? Qu’advient-il alors des données ?
- Où se trouve la clé de données en clair après l’exécution du script, et pourquoi est-ce important ?

5 Verrouiller un stockage objet et ranger un secret

1. Créez le bucket. Remplacez BINOME par vos initiales, **en minuscules** : un nom de bucket n’accepte que minuscules, chiffres et tirets. L’émulateur est permissif sur ce point, le vrai service ne l’est pas.

```
aws s3api create-bucket --bucket acme-rapports-BINOME --create-bucket-configuration
  LocationConstraint=eu-west-3
```

2. Appliquez les quatre verrous d’exposition publique, puis relisez-les :

```
aws s3api put-public-access-block --bucket acme-rapports-BINOME --public-access-block-
  configuration BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,
  RestrictPublicBuckets=true
aws s3api get-public-access-block --bucket acme-rapports-BINOME
```

3. Activez le versionnage, qui protège contre l’écrasement et la suppression :

```
aws s3api put-bucket-versioning --bucket acme-rapports-BINOME --versioning-configuration
  Status=Enabled
```

4. Écrivez `tls-obligatoire.json`, politique de ressource qui *refuse* tout accès non chiffré, en vous inspirant du modèle de l’annexe D, puis appliquez-la :

```
aws s3api put-bucket-policy --bucket acme-rapports-BINOME --policy file://tls-obligatoire.
  json
aws s3api get-bucket-policy --bucket acme-rapports-BINOME
```

5. Rangez le mot de passe de la base plutôt que de l’écrire dans le code :

```
aws secretsmanager create-secret --name prod/db/mot-de-passe --secret-string "P@ssw0rd-TP2"
aws secretsmanager get-secret-value --secret-id prod/db/mot-de-passe
```

6. Questions à traiter :

- La politique de la question 4 utilise `Deny`. Pourquoi un `Deny` et non l’absence d’`Allow` ?
- Le blocage d’accès public et la politique de bucket font-ils double emploi ? Lequel protège contre quoi ?
- Le secret est maintenant dans un coffre, mais l’application doit encore s’authentifier pour le lire. Avec quel mécanisme du TP 1 ?

Annexe A : fiche de relevé des identifiants

Ressource	Identifiant relevé
VPC	
Internet Gateway	
Subnet <code>public-a</code>	
Subnet <code>app-a</code>	
Subnet <code>data-a</code>	
Table de routage publique	
Table de routage privée	
Groupe <code>sg-alb</code>	
Groupe <code>sg-app</code>	
Groupe <code>sg-db</code>	
Identifiant de la clé KMS	

Annexe B : audit_reseau.py à compléter

```
import sys
import boto3

if len(sys.argv) != 2:
    print("usage : python audit_reseau.py vpc-xxxxxxx")
    sys.exit(1)

vpc = sys.argv[1]
ec2 = boto3.client("ec2")
f = [{"Name": "vpc-id", "Values": [vpc]}]

noms = {}
for s in ec2.describe_subnets(Filters=f)["Subnets"]:
    nom = "(sans nom)"
    for t in s.get("Tags", []):
        if t["Key"] == "Name":
            nom = t["Value"]
    noms[s["SubnetId"]] = nom

print("--- Subnets exposes : route 0.0.0.0/0 vers une Internet Gateway ---")
associes = set()
for rt in ec2.describe_route_tables(Filters=f)["RouteTables"]:
    expose = False
    for r in rt["Routes"]:
        cible = str(r.get("GatewayId", ""))
        # A COMPLETER : mettre expose a True si la destination est 0.0.0.0/0
        # et si la cible commence par "igw-"
    for a in rt.get("Associations", []):
        sid = a.get("SubnetId")
        if sid:
            associes.add(sid)
            print(" %-10s %-28s %s" % (noms.get(sid, "?"), sid,
                                     "EXPOSE" if expose else "prive"))

for sid, nom in noms.items():
    if sid not in associes:
        print(" %-10s %-28s %s" % (nom, sid,
                                   "table principale par default : a verifier"))

print("--- Regles ouvertes a Internet sur un port d'administration ---")
trouve = False
for sg in ec2.describe_security_groups(Filters=f)["SecurityGroups"]:
    for p in sg["IpPermissions"]:
        for r in p.get("IpRanges", []):
            # A COMPLETER : signaler la regle si r["CidrIp"] vaut "0.0.0.0/0"
            # et si p.get("FromPort") est 22 ou 3389
            pass
if not trouve:
    print(" aucune")
```

Annexe C : coffre.py

```
import os
import boto3
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

kms = boto3.client("kms")
message = b"salaires 2026 : confidentiel"

# 1. Demander une cle de donnees : en clair et chiffree par la cle maitresse
reponse = kms.generate_data_key(KeyId="alias/tp2", KeySpec="AES_256")
dek, dek_chiffree = reponse["Plaintext"], reponse["CiphertextBlob"]
print("DEK en clair :", len(dek), "octets")
print("DEK chiffree :", len(dek_chiffree), "octets")

# 2. Chiffrer localement, puis ecrire le coffre : DEK chiffree + nonce + donnees
nonce = os.urandom(12)
donnees = AESGCM(dek).encrypt(nonce, message, None)
open("coffre.bin", "wb").write(dek_chiffree + b"||" + nonce + b"||" + donnees)
del dek # la cle en clair ne survit pas a l'execution

# 3. Relire : seul le KMS peut redonner la DEK en clair
contenu = open("coffre.bin", "rb").read()
chiffree, nonce2, donnees2 = contenu.split(b"||", 2)
dek2 = kms.decrypt(CiphertextBlob=chiffree)["Plaintext"]
print("Dechiffre :", AESGCM(dek2).decrypt(nonce2, donnees2, None).decode())
```

Annexe D : modèle de politique de ressource

Adaptez le nom du bucket, puis complétez la condition pour qu'elle ne déclenche que sur les accès non chiffrés.

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "RefuserSansTLS",
    "Effect": "Deny",
    "Principal": "*",
    "Action": "s3:*",
    "Resource": [
      "arn:aws:s3:::acme-rapports-BINOME",
      "arn:aws:s3:::acme-rapports-BINOME/*"
    ],
    "Condition": { "Bool": { "aws:SecureTransport": "___" } }
  }]
}
```

Annexe E : mémo des commandes

```
# Environnement (terminal 1, a laisser tourner)
moto_server -p 5055
aws sts get-caller-identity # verification, terminal 2

# Reseau
aws ec2 create-vpc --cidr-block 10.0.0.0/16
aws ec2 create-subnet --vpc-id vpc-xxxx --cidr-block 10.0.0.0/24 --availability-zone eu-west-3a
aws ec2 create-tags --resources subnet-xxxx --tags Key=Name,Value=public-a
aws ec2 create-internet-gateway
aws ec2 attach-internet-gateway --internet-gateway-id igw-xxxx --vpc-id vpc-xxxx
aws ec2 create-route-table --vpc-id vpc-xxxx
aws ec2 create-route --route-table-id rtb-xxxx --destination-cidr-block 0.0.0.0/0 --gateway-id igw-xxxx
aws ec2 associate-route-table --route-table-id rtb-xxxx --subnet-id subnet-xxxx
aws ec2 describe-route-tables --filters Name=vpc-id,Values=vpc-xxxx

# Filtrage
aws ec2 create-security-group --group-name NOM --description "TEXTE" --vpc-id vpc-xxxx
aws ec2 authorize-security-group-ingress --group-id sg-xxxx --protocol tcp --port 443 --cidr 0.0.0.0/0
aws ec2 authorize-security-group-ingress --group-id sg-xxxx --protocol tcp --port 8443 --source-group sg-yyyy
aws ec2 revoke-security-group-ingress --group-id sg-xxxx --protocol tcp --port 22 --cidr 0.0.0.0/0
aws ec2 describe-security-groups --filters Name=vpc-id,Values=vpc-xxxx

# Donnees
aws s3api create-bucket --bucket NOM --create-bucket-configuration LocationConstraint=eu-west-3
aws s3api get-public-access-block --bucket NOM
aws s3api get-bucket-policy --bucket NOM
aws kms create-key --description "TEXTE"
aws kms create-alias --alias-name alias/tp2 --target-key-id KEY-ID
aws kms generate-data-key --key-id alias/tp2 --key-spec AES_256
aws s3api put-public-access-block --bucket NOM --public-access-block-configuration BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true
aws s3api put-bucket-versioning --bucket NOM --versioning-configuration Status=Enabled
aws s3api put-bucket-policy --bucket NOM --policy file://tls-obligatoire.json
aws secretsmanager create-secret --name prod/db/mot-de-passe --secret-string "VALEUR"
aws secretsmanager get-secret-value --secret-id prod/db/mot-de-passe
```